

UNIVERSIDAD NACIONAL DE LOJA



1859



Universidad
Nacional
de Loja

Carrera: Ingeniería en Computación

Asignatura: Teoría de la Programación

Docente: Lisete Geoconda López Faican

Ciclo: 1

Unidad: 2

Tema: Aplicación de estructuras condicionales y repetitivas en el diseño

Nombre: Matias Joel Ortega Correa

Fecha: 09/06/2026

- **CONTENIDOS**

Estructuras condicionales

Las estructuras condicionales permiten que un programa tome decisiones según una condición. Es decir, el programa evalúa si algo es verdadero o falso y, dependiendo del resultado, ejecuta una acción.

Tipos principales:

1. Condicional simple

Se usa cuando solo se ejecuta una acción si la condición se cumple.

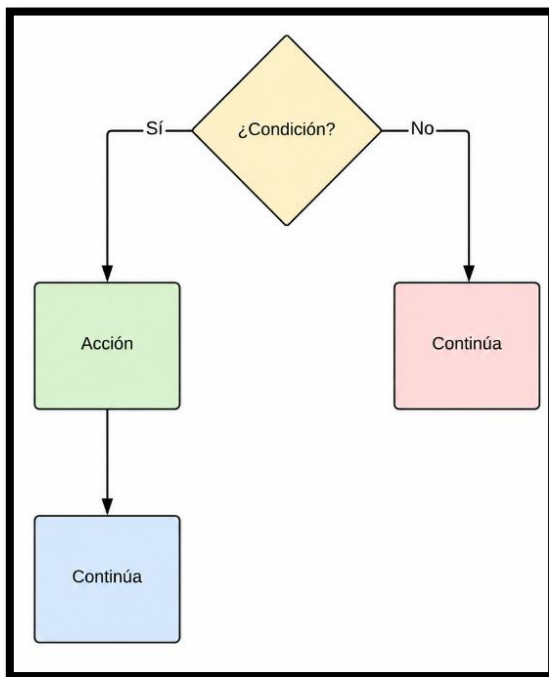
Pseudocódigo:

Si condición Entonces

 Acción

FinSi

Diagrama de Flujo:



2. Condicional doble

Se usa cuando hay una acción si la condición es verdadera y otra si es falsa.

Pseudocódigo:

Si condición Entonces

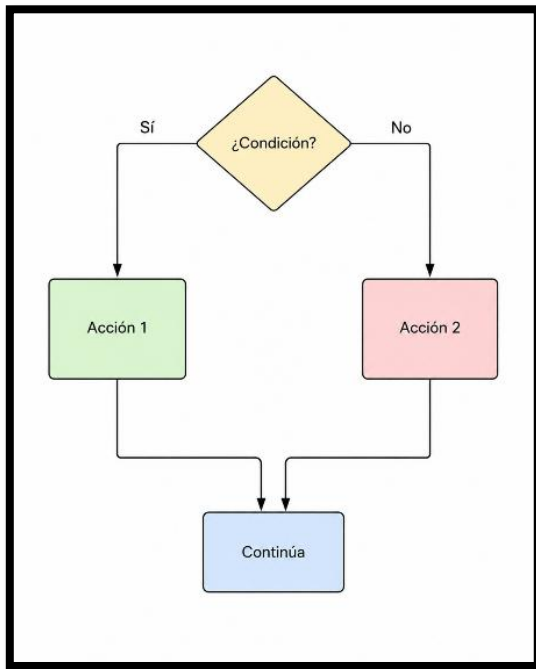
Acción 1

Sino

Acción 2

FinSi

Diagrama de Flujo:



3. Condicional multiple

Se usa cuando existen varias condiciones o varios casos posibles.

Pseudocódigo:

Según variable Hacer

Caso 1:

Acción 1

Caso 2:

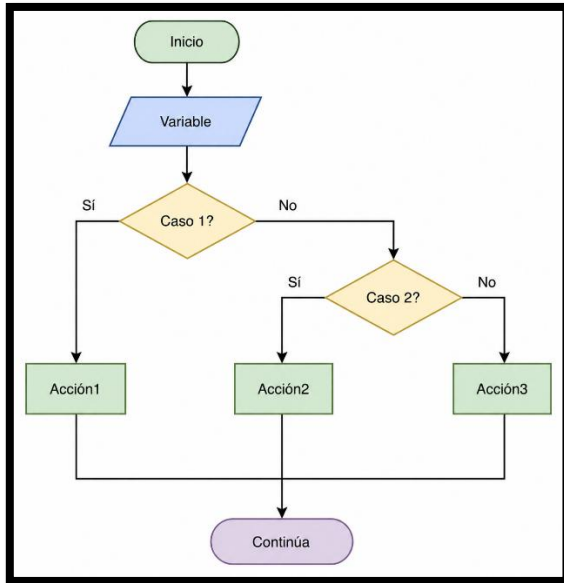
Acción 2

De Otro Modo:

Acción 3

FinSegún

Diagrama de flujo:



Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar una o varias instrucciones varias veces, mientras se cumpla una condición o hasta completar un número de repeticiones.

1. Mientras

Se usa cuando primero se evalúa la condición y luego se ejecutan las instrucciones.

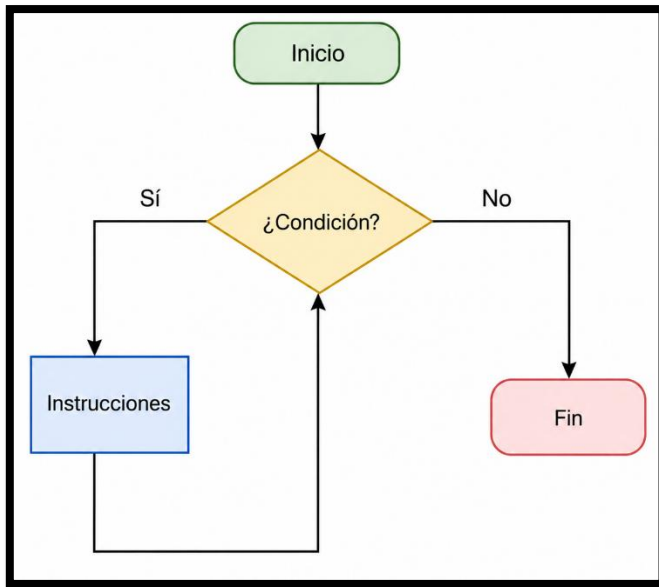
Pseudocódigo:

Mientras condición Hacer

 Instrucciones

FinMientras

Diagrama de flujo:



2. Repetir...Hasta Que

Se ejecuta al menos una vez y luego evalúa la condición.

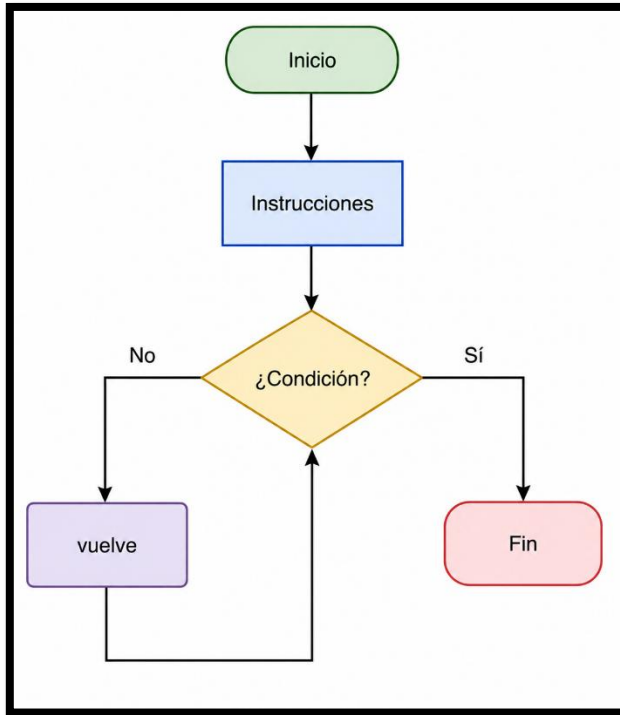
Pseudocódigo:

Repetir

 Instrucciones

Hasta Que condición

Diagrama de Flujo:



3. Para

Se usa cuando se conoce la cantidad de repeticiones.

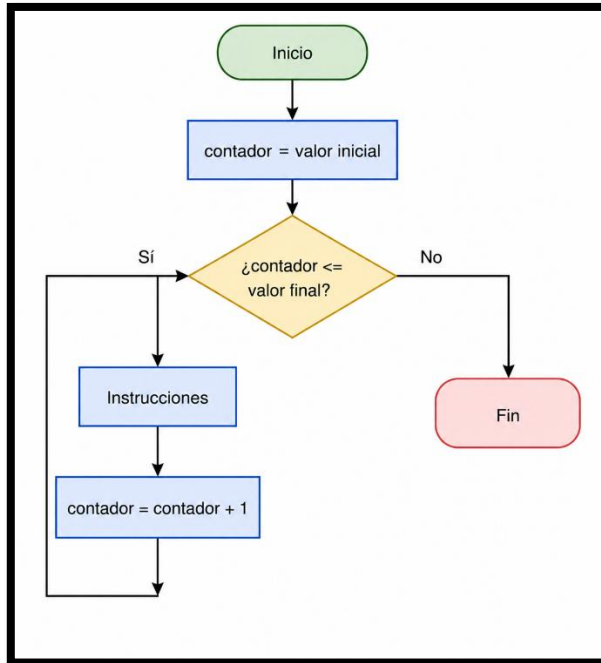
Pseudocódigo:

Para contador <- valorInicial Hasta valorFinal Hacer

 Instrucciones

FinPara

Diagrama de flujo:



Ejercicio: Control de ventas en una tienda

Planteamiento del problema

Se desea crear un programa que permita ingresar la cantidad de productos vendidos en una tienda. Por cada producto se debe ingresar su precio. Si el precio es menor o igual a 0, el programa debe mostrar un mensaje de error y volver a pedir el dato.

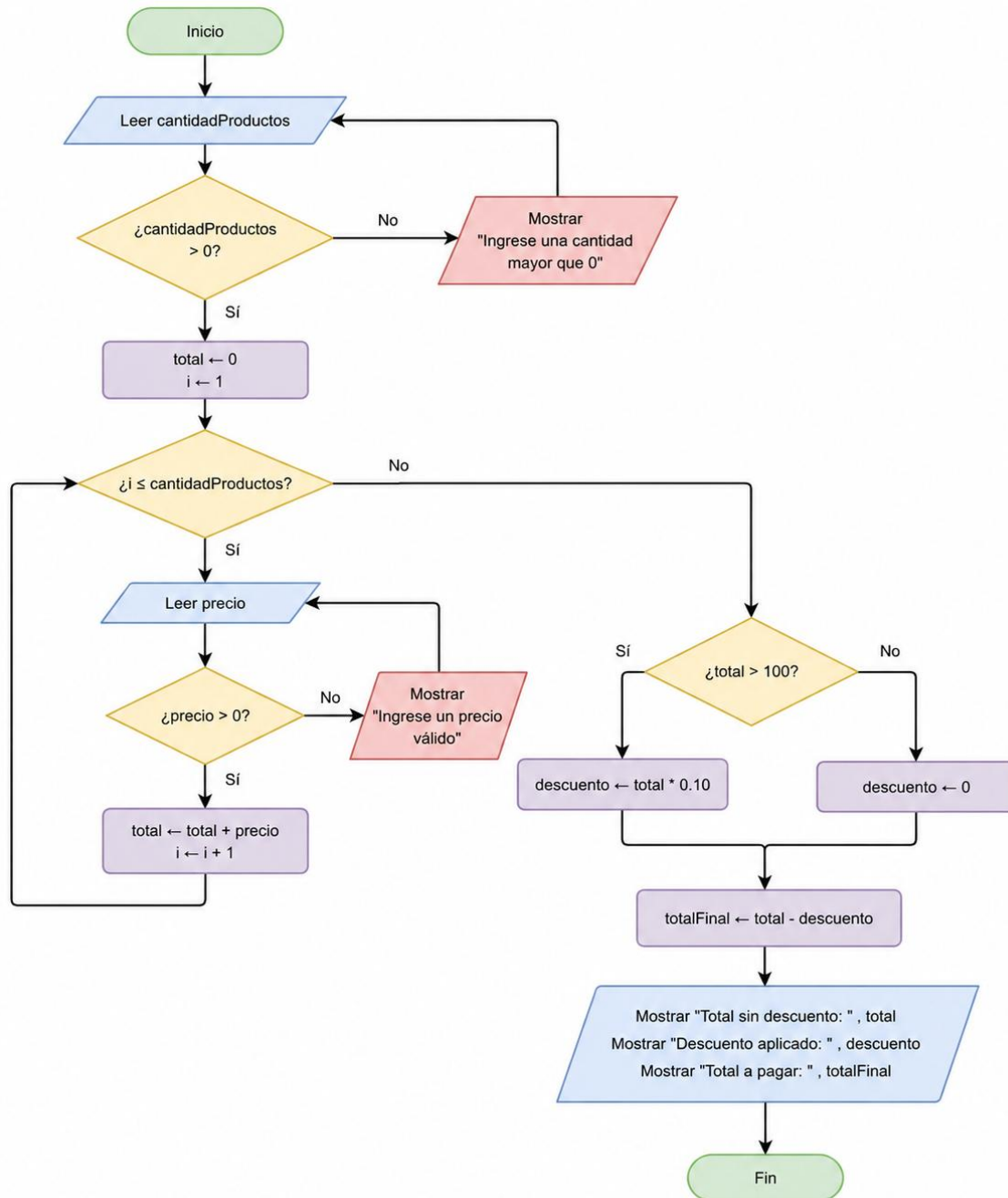
Al final, el programa debe calcular el total de la venta. Si el total supera los \$100, se aplica un descuento del 10%. Caso contrario, no se aplica descuento.

Análisis del problema

Elemento	Descripción
Datos de entrada	Cantidad de productos vendidos, precio de cada producto.
Proceso	1. Solicitar la cantidad de productos. 2. Validar que los productos sean mayores que 0 3. Solicitar el precio de cada producto 4. Validar que cada precio sea mayor que 0. 5. Acumular el total de las ventas. 6. Verificar si el total supera los \$100.

	7. Si supera los \$100, calcular un descuento del 10%; caso contrario, descuento igual a 0. 8. Calcular el total final a pagar.
Datos de salida	Total de la venta, descuento aplicado y total final a pagar.

Diseño del algoritmo (Diagrama de Flujo)



Codificación

```
C totalProductos.c
1  #include <stdio.h>
2
3  int main() {
4
5      int cantidadProductos, i;
6      float precio, total = 0;
7      float descuento, totalFinal;
8
9      // Ingreso y validación de la cantidad de productos
10     do {
11         printf("Ingrese la cantidad de productos vendidos: ");
12         scanf("%i", &cantidadProductos);
13
14         if (cantidadProductos <= 0) {
15             printf("Ingrese una cantidad mayor que 0\n");
16         }
17     } while (cantidadProductos <= 0);
18
19     // Ingreso y validación de precios
20     for (i = 1; i <= cantidadProductos; i++) {
21
22         do {
23             printf("Ingrese el precio del producto #%i: ", i);
24             scanf("%f", &precio);
25
26             if (precio <= 0) {
27                 printf("Ingrese un precio valido\n");
28             }
29         } while (precio <= 0);
30
31         total = total + precio;
32     }
33
34     // Aplicación del descuento
35     if (total > 100) {
36         descuento = total * 0.10;
37     } else {
38         descuento = 0;
39     }
40
41     totalFinal = total - descuento;
42 }
```

```

43     // Cálculo del total final
44     totalFinal = total - descuento;
45
46     // Resultados
47     printf("\nTotal sin descuento: $%.2f\n", total);
48     printf("Descuento aplicado: $%.2f\n", descuento);
49     printf("Total a pagar: $%.2f\n", totalFinal);
50
51     return 0;
52 }
53

```

Validación (Prueba de escritorio)

Paso	Dato ingresado	Validación	Total acumulado
Cantidad de productos	3	Válido	0
Producto #1	40	Válido	40
Producto #2	-10	Inválido, vuelve a pedir	40
Producto #2	35	Válido	75
Producto #3	50	Válido	125

Concepto	Valor
Total de venta	\$125.00
¿Total > 100?	Sí
Descuento (10%)	\$12.50
Total a pagar	\$112.50

```
Ingrese la cantidad de productos vendidos: 3
Ingrese el precio del producto #1: 40
Ingrese el precio del producto #2: -10
Ingrese un precio valido
Ingrese el precio del producto #2: 35
Ingrese el precio del producto #3: 50

Total sin descuento: $125.00
Descuento aplicado: $12.50
Total a pagar: $112.50
```

Principales dificultades

Una de las principales dificultades fue identificar correctamente cuándo usar una estructura condicional y cuándo usar una estructura repetitiva. También fue necesario validar los datos de entrada, ya que el programa no debe aceptar cantidades o precios menores o iguales a cero. Otra dificultad fue acumular correctamente el total de la venta dentro del ciclo, porque si esta operación se coloca fuera del for, el programa no sumaría todos los productos.

Reflexión crítica

La aplicación de estructuras condicionales y repetitivas permite resolver problemas de manera más ordenada y eficiente. En este ejercicio, las estructuras repetitivas ayudaron a ingresar varios productos sin repetir muchas veces el mismo código, mientras que las condicionales permitieron validar los datos y aplicar el descuento según el total de la compra. Esto demuestra que antes de programar es importante analizar el problema y diseñar el algoritmo, ya que así se reducen errores y se comprende mejor la lógica del programa.